

KrishiNirnay

Technical Approach & System Architecture

Farmer Decision-Support Android Application

1. Technical Approach

1.1 Problem Approach

KrishiNirnay is a farmer-facing decision-support Android application that combines real-time weather, mandi (market) prices, crop disease/pest detection, and an AI assistant into a single tool, built around three non-negotiable principles enforced throughout the codebase:

1. **Never fabricate data.** Every data-driven screen (Weather, Market, Disease, AI Assistant) has an explicit "unavailable"/"no data" state. If a real reading can't be fetched, the UI says so honestly instead of showing a plausible-looking fake number.
2. **Prefer on-device / cached over network-dependent.** Where a genuine on-device or cached alternative exists (AI inference, last-known weather/market reading), it is preferred over a network call, so the app degrades gracefully rather than freezing or blanking out.
3. **Real device verification over assumption.** Every major feature claim in this project was checked against actual behavior — real device screenshots, real memory measurements, real API responses — rather than treated as "should work."

1.2 Technology Stack

Client (Android app)

- Kotlin, Jetpack Compose (declarative UI), Material 3
- MVVM architecture: `ViewModel` + `StateFlow` / `Flow` for unidirectional state
- Hilt (Dagger) for dependency injection across the whole app
- Coroutines for all async work (network calls, DB, sensors, AI generation)
- Retrofit + OkHttp + `kotlinx.serialization` for networking
- Jetpack DataStore for local persistence/caching (weather, market, field-state caches)
- Firebase Authentication + Cloud Firestore for account/profile/farm data
- Google LiteRT-LM (`com.google.ai.edge.litertlm`) for on-device LLM inference

Backend (FastAPI server)

- Python, FastAPI, Uvicorn (ASGI)
- Pydantic / pydantic-settings for schema validation and typed configuration
- scikit-learn (Agricultural Risk model, joblib-pickled)
- `ai-edge-litert` (LiteRT/TFLite interpreter) for Disease Detection inference
- `onnxruntime` for Pest Detection inference (custom YOLO decode + NMS)
- httpx for outbound calls to third-party APIs (WeatherAPI.com, data.gov.in AGMARKNET, Ollama)

- Deployed as a Docker container on Render (free tier, 512MB RAM)

AI/ML models

- Disease Detection: CNN classifier (23 crop-disease classes), trained in Keras, converted to TFLite for deployment
- Pest Detection: YOLOv8 object detector (10 pest classes), trained in PyTorch/Ultralytics, exported to ONNX for deployment
- Agricultural Risk: scikit-learn classifier fusing crop/soil/weather features into a 3-class risk level
- Local LLM: Gemma 3 1B (int4 quantized) for on-device chat; Ollama-hosted models (deepseek-r1:7b default, qwen2.5:3b/7b for Hindi/Marathi) as the server-side fallback

1.3 Design Approach

- **Repository pattern with a Mock/Live switch.** Every data source (Weather, Market, FieldState) has a `Mock` implementation (offline, procedurally generated demo data) and a `Live` implementation (real API-backed), bound behind a single interface via Hilt, switched at runtime from Settings. This lets development/demo continue with zero network dependency while production farmers get real data.
- **Provider-coordinator pattern for AI.** A single `AiProviderCoordinator` abstracts "where does the AI answer come from" behind one interface, trying the on-device model first and falling back to the server only if on-device isn't ready — the calling code (Chatbot UI) never needs to know which path served the answer.
- **Honest status enums, not booleans.** Data source status is modeled as `DataSourceStatus { LIVE, CACHED, LOADING, NO_DATA, UNAVAILABLE }` rather than a simple success/failure flag, so the UI can distinguish "fresh real data," "stale but real data," and "genuinely nothing" — and render each differently instead of collapsing them into one generic error state.
- **Cache-then-degrade, never crash.** Weather and Market repositories persist the last successful reading to DataStore. On a failed refresh they fall back to that cached reading (marked `CACHED`) rather than clearing the screen, and only show `UNAVAILABLE` if nothing has ever been fetched.
- **Verify before claiming.** Architectural or performance decisions in this project (model format conversions, memory budgets, hosting-plan sizing) were validated with real, reproducible measurements (numerical equivalence tests, live memory profiling, on-device screenshots) rather than accepted on assumption.

2. System Architecture

2.1 High-Level Architecture



2.2 Client-Side Architecture (Android)

Layered structure:

- **UI layer** — Jetpack Compose screens (Home, Weather, Market, Disease, AI Assistant/Chatbot, Profile, Farm Setup, Settings, Login/Register), each observing one `ViewModel`'s `StateFlow<UiState>`.
- **ViewModel layer** — one `@HiltViewModel` per screen, orchestrating repository calls, holding UI state, exposing user actions (`sendMessage()` , `cancelGeneration()` , `refreshWeather()` , etc.). No business logic in Composables.
- **Repository layer** — interfaces (`WeatherRepository` , `MarketRepository` , `FieldStateRepository` , `ProfileRepository` , `LocalLlmRepository` , `AuthRepository`) with swappable implementations bound via Hilt `@Binds`.
- **Data sources:**
 - Firebase Auth + Firestore — account creation/login, farmer profile, farm setup (crop, state, district, village, land area, soil, irrigation).
 - Retrofit/OkHttp — talks to the FastAPI server for Weather, Market, Disease, Pest, Risk, and the server-side LLM fallback.
 - DataStore-backed caches (`WeatherStateCache` , `MarketStateCache` , `FieldStateCache`) — persist the last successful reading across app restarts.
 - LiteRT-LM Engine (`MediaPipeOnDeviceLlmProvider`) — runs the on-device Gemma model directly on the phone's CPU, no network required.

Cross-cutting concerns:

- `AiProviderCoordinator` — single source of truth for "which AI provider is active" (`ON_DEVICE` , `SERVER` , or `NONE`), exposed as `StateFlow` so any screen can reactively show the right status.
- `OnDeviceModelManager` — handles the farmer-triggered model download: storage-space pre-check, progress reporting, corruption detection (declared vs. actual byte-count verification), and cleanup.
- `AppStrings` — centralized, language-keyed (English/Hindi/Marathi) string resolution for every user-facing message, including every "unavailable"/error string, so honesty-by-design extends to every supported language.

2.3 Server-Side Architecture (FastAPI)

Routers (each independently mountable, each behind an API-key dependency):

Router	Responsibility	Backing model/API
weather	Current + forecast weather for a farmer's location	WeatherAPI.com
market	Mandi prices for a crop (state + district preference)	data.gov.in AGMARKNET


```

|
|                                     zero network calls)
+-- No --> LocalLlmRepositoryImpl (server path)
|
| v
| FastAPI /v1/local-llm/ask
|
| v
| Ollama (self-hosted LLM server)

```

- `AIProviderCoordinator` maintains a `StateFlow<AIProviderKind>` (`ON_DEVICE` / `SERVER` / `NONE`) and a `StateFlow<LocalLlmStatus>` (`LOADING` / `READY` / `MODEL_MISSING` / `UNAVAILABLE` / `GENERATING` / `ERROR`), refreshed on app start, after a model download completes, and whenever the Chat screen opens.
- If neither path is available, the UI shows one exact, honest message: *"Local AI is unavailable. Download the on-device AI model or connect to the AI server."* — never a fabricated answer.
- Generation is cancellable mid-stream (coroutine cancellation propagates down to the `LiteRTConversation.cancelProcess()` call), and cancelling never leaves a "stuck typing" state or a zombie generation if a new question is asked immediately after.
- The same structured farmer context (state, district, crop, soil, irrigation, current weather/field-state) is built once (`LocalLlmContextBuilder`) and fed identically to both the on-device and server paths, so answer quality doesn't depend on which provider happened to answer.

2.5 Data Flow — Farmer Profile to Every Feature

```

Register/Login (Firebase Auth)
|
| v
Farmer Profile + Farm Setup (name, mobile, state, district, village,
|                           language, crop, land area, soil, irrigation)
|   saved to Firestore + local cache
|
+--> Weather:      state/district/village -> location lookup -> WeatherAPI.com
|
+--> Market:       crop + state + district (district = preference,
|                           not a hard filter) -> AGMARKNET price lookup
|
+--> Disease/Pest: selected crop context + uploaded photo -> CNN/YOLO
|
+--> AI Assistant: full farmer/farm context + field state + weather
|                           + market -> grounded prompt -> on-device or server LLM

```

No screen uses hardcoded/demo values once a real farmer profile exists — every one of the above reads the farmer's actual saved data, and every one has an honest empty/unavailable state if that data (or the underlying API) isn't available.

2.6 Deployment Architecture

- **Android app:** distributed as a debug/release APK; Hilt wires all dependencies at compile time; no server-side rendering or dynamic feature delivery.
- **Backend:** single Docker image (`python:3.11-slim` base) containing the FastAPI app, the two lightweight ML runtimes, and the two exported model artifacts (`best.onnx` , `disease_model_finetuned_best.tflite`) — deployed to Render as a Blueprint (`render.yaml`) on the free web-service plan, health-checked at `/health` .
- **Secrets:** API key, Gemini key, WeatherAPI key, and data.gov.in key are all injected as Render environment variables (`sync: false`), never committed to the repository.
- **On-device model distribution:** the licensed Gemma model file is never bundled in the app or the server's git repo (Google's license requires manual acceptance); the server exposes a download endpoint that 404s honestly until a developer manually places the file, and the Android app only downloads it on an explicit farmer tap in Settings — never automatically.